



# VERIFIZIERTE C-LIBRARY FÜR LOCKFREIE DATENSTRUKTUREN

## ENTWICKLUNG DER LOCKFREIEN DATENSTRUKTUREN

Software-Entwicklungspraktikum (SEP)

Sommersemester 2023

### Fachentwurf

Auftraggeber

Technische Universität Braunschweig

Institute of Theoretical Computer Science

Prof. Dr. rer. nat. Roland Meyer

IZ 346, i.e. Informatikzentrum, 3rd floor, office 346, Mühlenpfordtstr. 23

D-38106 Braunschweig

Betreuer: Sören van der Wall

Auftragnehmer:

| Name                 | E-Mail-Adresse                    |
|----------------------|-----------------------------------|
| Keanu Wirz           | k.wirz@tu-braunschweig.de         |
| Nazli Cesur          | n.cesur@tu-braunschweig.de        |
| Tristen Vaeckenstedt | t.vaeckenstedt@tu-braunschweig.de |
| Jacob Müller         | jacob.mueller@tu-braunschweig.de  |
| Mohamed Amine Halila | m.halila@tu-braunschweig.de       |
| Omar Zitouni         | o.zitouni@tu-braunschweig.de      |

Braunschweig, 7. Juni 2023

## Bearbeiterübersicht

| Kapitel | Autoren                    | Kommentare |
|---------|----------------------------|------------|
| 1       | Nazli                      | Jacob      |
| 1.1     | Nazli                      | Tristen    |
| 2       | Omar, Keanu, Mohamed Amine | Jacob      |
| 3       | Tristen                    | Nazli      |
| 4       | Jacob                      | Keanu      |
| 5       | Alle                       | Alle       |

## Inhaltsverzeichnis

|          |                                                             |           |
|----------|-------------------------------------------------------------|-----------|
| <b>1</b> | <b>Einleitung</b>                                           | <b>5</b>  |
| 1.1      | Projektdetails . . . . .                                    | 8         |
| 1.1.1    | Compare-and-Swap Funktion . . . . .                         | 8         |
| 1.1.2    | Benchmark und Visualisierung . . . . .                      | 9         |
| 1.1.3    | Deallokation der Nodes . . . . .                            | 9         |
| <b>2</b> | <b>Analyse der Produktfunktionen</b>                        | <b>10</b> |
| 2.1      | Analyse von Funktionalität <F10>: <Push im Stack> . . . . . | 10        |
| 2.2      | Analyse von Funktionalität <F20>: <Pop aus Stack> . . . . . | 10        |
| 2.3      | Analyse von Funktionalität <F30>: <Enqueue> . . . . .       | 10        |
| 2.4      | Analyse von Funktionalität <F40>: <Dequeue> . . . . .       | 11        |
| 2.5      | Analyse von Funktionalität <F50>: <Insert> . . . . .        | 11        |
| 2.6      | Analyse von Funktionalität <F60>: <Delete> . . . . .        | 11        |
| 2.7      | Analyse von Funktionalität <F70>: <Find> . . . . .          | 12        |
| 2.8      | Analyse von Funktionalität <F80>: <Search> . . . . .        | 12        |
| 2.9      | Analyse von Funktionalität <F90>: <Visualisieren> . . . . . | 13        |
| 2.10     | Analyse von Funktionalität <F100>: <Benchmark> . . . . .    | 13        |
| <b>3</b> | <b>Datenmodell</b>                                          | <b>24</b> |
| 3.1      | Diagramm . . . . .                                          | 24        |
| 3.2      | Erläuterung . . . . .                                       | 24        |
| <b>4</b> | <b>Konfiguration</b>                                        | <b>25</b> |
| <b>5</b> | <b>Glossar</b>                                              | <b>26</b> |

## Abbildungsverzeichnis

|      |                                                               |    |
|------|---------------------------------------------------------------|----|
| 1.1  | Gesamter Anwendungsprozess als Aktivitätsdiagramm . . . . .   | 7  |
| 1.2  | CAS Aktivitätsdiagramm . . . . .                              | 8  |
| 1.3  | Freigabe der Speicheradresse als Aktivitätsdiagramm . . . . . | 9  |
| 2.1  | Pushen im Stack . . . . .                                     | 14 |
| 2.2  | Pop aus Stack . . . . .                                       | 15 |
| 2.3  | Enqueue . . . . .                                             | 16 |
| 2.4  | Dequeue . . . . .                                             | 17 |
| 2.5  | Insert im Set . . . . .                                       | 18 |
| 2.6  | Delete . . . . .                                              | 19 |
| 2.7  | Find . . . . .                                                | 20 |
| 2.8  | Search in Set . . . . .                                       | 21 |
| 2.9  | Visualisieren . . . . .                                       | 22 |
| 2.10 | Benchmark . . . . .                                           | 23 |
| 3.1  | Diagramm zur Erstellung der Benchmark Ergebnisse . . . . .    | 24 |

# 1 Einleitung

Dieses Dokument dient als Fachentwurf für das Projekt lockfreie Datenstrukturen, welches dem Auftraggeber ein Überblick über die Funktionen des Produkts verschaffen soll.

Als erstes wird ein Aktivitätsdiagramm gegeben, welches über das Softwaresystem eine Übersicht gibt. Im Kapitel 1 werden die komplizierten und wichtigen Prozesse von unserer Software aufgelistet und diese detailliert erklärt. In Kapitel 2 wird das Verhalten von Produktfunktionen mit Sequenzdiagrammen dargestellt und erklärt. Im Schluss behandeln wir die geplante Konfiguration der Software, einschließlich der technischen Umgebung, der erforderlichen Hardware- und Softwarekomponenten.

Das Ziel dieses Projekts ist es, eine lockfreie Library für die Programmiersprache C zu entwickeln. Die Library bietet eine effiziente und fehlerfreie Möglichkeit, Synchronisationsmechanismen in Mehrkernsystemen zu implementieren, ohne auf Sperren (Locks) zurückgreifen zu müssen. Durch den Verzicht auf Locks wird das Problem von Wettlaufsituationen (Race Conditions) und Deadlocks eliminiert, während gleichzeitig die Leistungsfähigkeit des Systems gesteigert wird. Wie in der Abbildung 1.1 beschrieben, ermöglicht unsere Visualisierung dem Nutzer, die Anzahl der Threads, sowie die Anzahl der Operationen anzugeben. Die Library bietet eine Reihe von Datenstrukturen: Queue, Stack und List, die mit den Prinzipien der Thread-Sicherheit und atomaren Operationen implementiert wurden. Die Datenstrukturen haben jeweils die Einfügeoperationen (push, enqueue, insert) und Löschoperationen (pop, dequeue, delete).

Beim Einfügen wird erst eine neue Node alloziert. Beim Set insert, anders als Stack und Queue, wird erst die richtige Stelle gesucht und gecheckt, ob das einzufügende Element schon im Set existiert. Wenn das Element schon existiert, ist nichts zu tun, die Funktion returnt 0. Wenn nicht, wird die aktuelle Datenstruktur, wie bei Queue und Stack gelesen. Danach wird der Node lokal initialisiert, wie es zur Datenstruktur passt. Es wird im Schluss mit der Compare and Swap Funktion (CAS) gecheckt, ob der gelesene Wert mit der aktuellen Datenstruktur immernoch übereinstimmen. Wenn sie übereinstimmen, heißt das, dass keine anderen Threads die Datenstruktur zwischendurch verändert haben. Der Thread darf das Element hinzufügen bzw. die Datenstruktur wird aktualisiert. Wenn sie nicht übereinstimmen, heißt das, dass ein anderer Thread die Datenstruktur modifiziert hat. Der Thread muss nochmal die Datenstruktur lesen und mit CAS checken.

Beim Löschen von Set, anders als Stack und Queue, wird erst das zu löschende Element gesucht, ob es überhaupt im Set existiert. Wenn das Element nicht gefunden wird, terminiert die Funktion. Wenn es gefunden wurde, wird der aktuelle Stack gelesen und die Datenstruktur wird lokal aktualisiert, sodass das Element entfernt wird. Es wird am Schluss mit der Compare and Swap Funktion gecheckt, ob der gelesene Wert mit der aktuellen Datenstruktur immer noch übereinstimmt. Wenn sie übereinstimmen, heißt das, dass kein andere Thread die Operation unterbrochen hat. Der Thread darf das Element hinzufügen bzw. die Datenstruktur aktualisiert sich als lokale Datenstruktur. Wenn sie nicht übereinstimmen, heißt das, dass ein andere Thread die Operation unterbrochen hat. Der Thread muss nochmal die Datenstruktur lesen und mit CAS checken.

Das Interface bietet auch die Möglichkeit, die Ergebnisse grafisch darzustellen. Nach Abschluss der Operationen generiert das Interface ein Geschwindigkeitsdiagramm, das die gemessene Leistung der lockfreien Datenstrukturlibrary in Bezug auf die angegebenen Parameter visualisiert. Dadurch kann der Benutzer die Effizienz und Skalierbarkeit der Library direkt beobachten und analysieren.

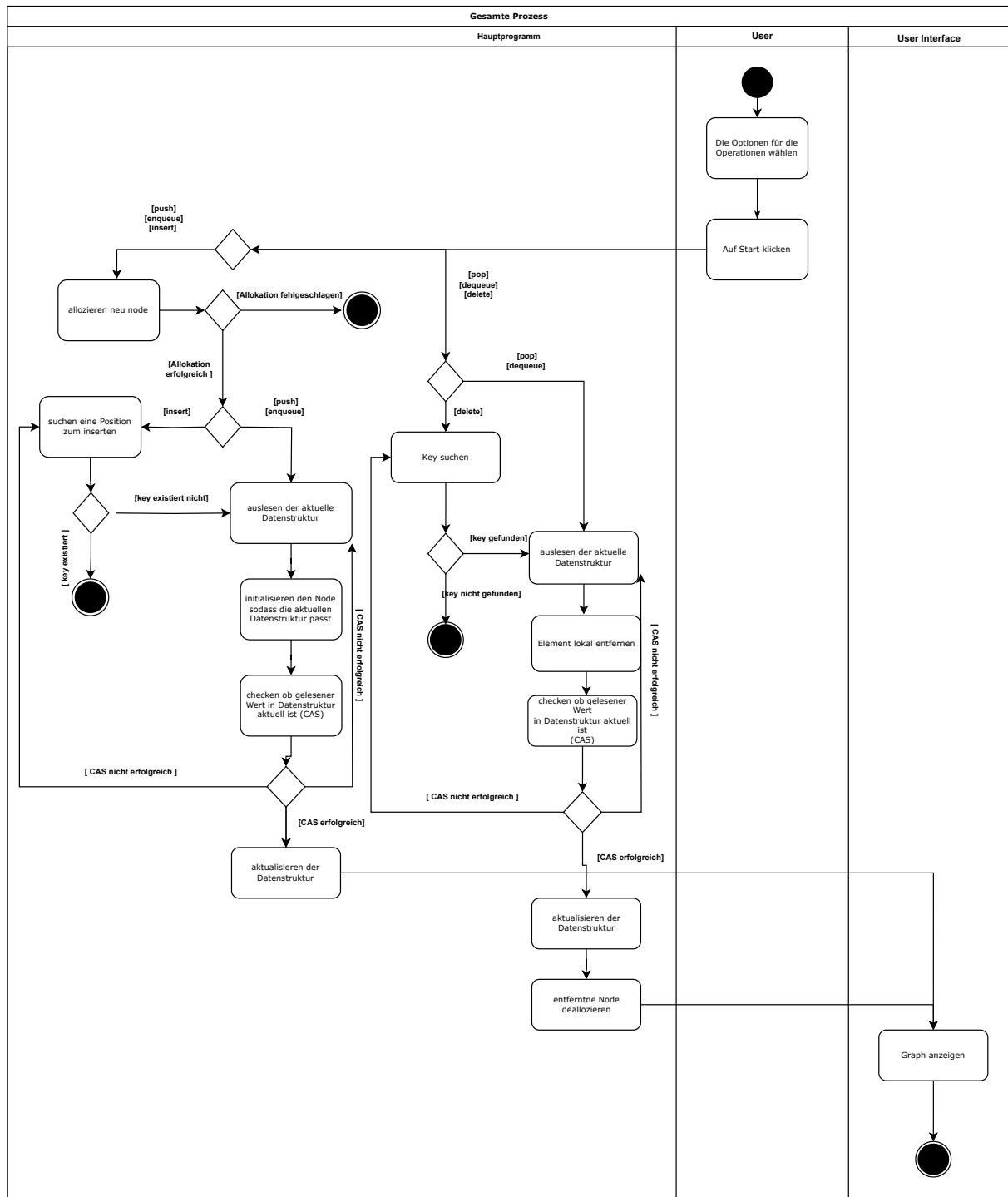


Abbildung 1.1: Gesamter Anwendungsprozess als Aktivitätsdiagramm

## 1.1 Projektdetails

In diesem Abschnitt werden komplizierte Sachverhalten detailliert erklärt, die in der Abbildung 1.1 nicht vertieft werden.

- Compare and Swap Funktion
- Benchmark und Visualisierung
- Deallokation der Nodes

### 1.1.1 Compare-and-Swap Funktion

Diese Funktion wird benutzt um Daten atomar zu aktualisieren. Die grundlegende Funktionsweise von Compare-and-Swap besteht darin, den aktuellen Wert einer Speicherzelle mit einem erwarteten Wert zu vergleichen. Wenn der aktuelle Wert mit dem erwarteten Wert übereinstimmt, wird ein neuer Wert in die Speicherzelle geschrieben. Andernfalls wird keine Aktualisierung durchgeführt. Im Zusammenhang von unserem Projekt ist der CAS wichtig um in einer einzigen Operation Veränderungen vorzunehmen die nicht unterbrochen werden kann.

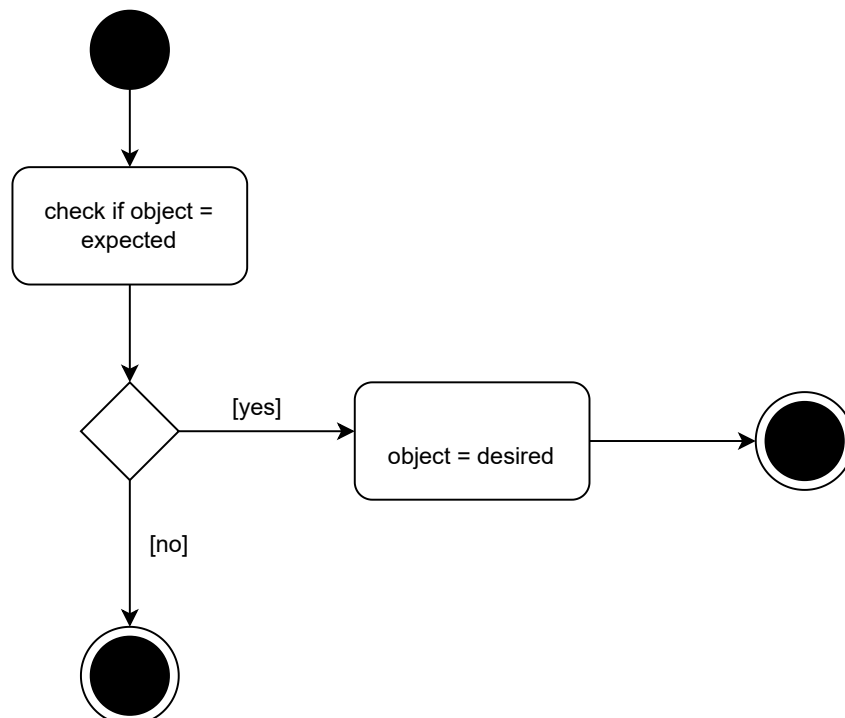


Abbildung 1.2: CAS Aktivitätsdiagramm



### 1.1.2 Benchmark und Visualisierung

Um die Effizienz und Leistungsfähigkeit der lockfreien Datenstrukturlibrary zu validieren, werden Benchmarks durchgeführt. Diese Benchmarks messen und vergleichen die Leistung der lockfreien Datenstrukturen mit herkömmlichen Lock-basierten Ansätzen. Die Ergebnisse werden in einem Benchmark-Diagramm präsentiert, das die Vorteile und Verbesserungen der lockfreien Implementierung verdeutlicht. Der Benchmark erstellt die gewünschte Anzahl der Threads, die dann die gewünschte Anzahl der Operationen ausführt und dann die Ergebnisse der Latenz und Overalltime Bestimmung in eine .csv Datei speichert, die dann das Visualisierungsprogramm auswertet werden. Für jeden Operationsaufruf wird Clock-Timer gestartet. Nachdem Aufruf fertig durchgeführt ist, wird Timer gestoppt. Damit berechnen wir die durchschnittliche Dauer von Datenstruktur Funktionen. Die Dauern wird temporär gespeichert und diese Werte in Graph angezeigt.

### 1.1.3 Deallokation der Nodes

Es kommt zu ungewünschtem Verhalten, wenn auf eine Speicheradresse zugegriffen wird, nachdem sie bereits freigegeben wird. Um das zu verhindern, wird ein Reference Counter benutzt. Vor dem freigeben einer Adresse, wird erst gecheckt ob gleiche Adresse von anderen Prozessen benutzt wird. Diese Überprüfung geschieht durch eine counter value innerhalb einer Node. Bei jedem Zugriff inkrementiert der counter und dekrementiert wenn die Adresse nicht mehr gebraucht ist. Wenn Counter auf 0 steht, kann der belegte Speicher freigegeben werden..

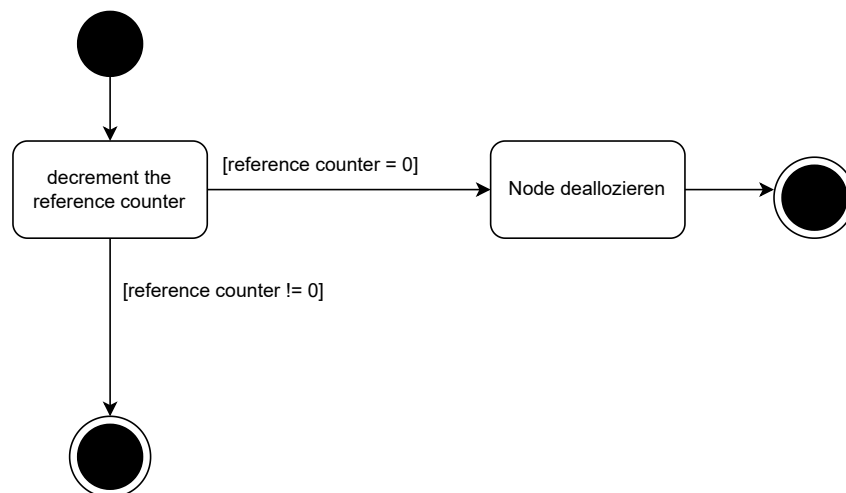


Abbildung 1.3: Freigabe der Speicheradresse als Aktivitätsdiagramm

## 2 Analyse der Produktfunktionen

Im folgenden Kapitel werden die im Pflichtenheft definierten Produktfunktionen anhand ihres Verhalten analysiert. Dies geschieht mit Sequenzdiagrammen, um die Kommunikation und die Interaktion von Objekten grafisch darzustellen.

### 2.1 Analyse von Funktionalität <F10>: <Push im Stack>

<F10>: In diesem Sequenzdiagramm, wird der Prozess des Push im Stack dargestellt. Dies geschieht indem der User die funktion `push(value)` aufruft. Hinterher werden lokale Objekte `newstack`, `oldstack` erstellt, um den alten und den neuen Stack zu halten. Danach wird ein neuer Knoten `newnode` im Speicher reserviert und der Wert `value` im `data`-feld des Knoten zugewiesen. Nachfolgend wird der globale Stack im `oldstack` durch atomares Laden gelesen und der `next` zeiger von der neuen Node wird auf den `top` von `oldstack` gesetzt. Anschließend wird `newstack` aktualisiert, indem sein `top` auf der neue Node `newnode` zeigt. Endlich wird der globalen Stack durch den neuen Stack `newstack` ersetzt.

### 2.2 Analyse von Funktionalität <F20>: <Pop aus Stack>

<F20>: In diesem Sequenzdiagramm, wird der Prozess des Pop aus Stack dargestellt. Dies geschieht in dem der User die funktion `pop(Stack)` aufruft. Hinterher werden lokale Objekte `newstack`, `oldstack` erstellt, um den alten und den neuen Stack zu halten. Dann wird geprüft ob die `top` Node von `oldstack` gleich `NULL` ist oder nicht. Falls ja, dann ist der Stack leer und die Funktion gibt `-1` zurück. Ansonsten wird "newstack" aktualisiert, indem sein `top` auf den `next` Knoten von "top des `oldstack`" zeigt. Endlich wird der globalen Stack durch ein neuer Stack ("newstack") ersetzt und der alte `top` Knoten des Stack wird freigegeben.

### 2.3 Analyse von Funktionalität <F30>: <Enqueue>

<F30>: In diesem Sequenzdiagramm, wird der Prozess Enqueue dargestellt. Dies geschieht indem der User die Funktion `enqueue(value)` aufruft. Hinterher wird ein neuer Knoten `newnode`

alloziert und dem der Wert `value` zugewiesen. Danach wird ein lokaler Zeiger `tail` initialisiert und dieser zeigt dann auf die aktuelle Adresse des globalen Zeigers `Tail`. Nachfolgend wird ein anderer Zeiger `nexttail` initialisiert und auf der Adresse von `next` von `tail` zeigen. Dann wird geprüft ob `tail` noch gleich zum globalen `Tail` ist und ob `nexttail` gleich `NULL` ist. Falls ja, dann werden die `next` von `tail` zu `newnode` und der globale `Tail` durch `newnode` ersetzt. Ansonsten wird der loop ab dem `atomicload` wiederholt bis `enqueue` erledigt ist.

## 2.4 Analyse von Funktionalität <F40>: <Dequeue>

<F40>: In diesem Sequenzdiagramm, wird der Prozess des Dequeue dargestellt. Dies geschieht indem der User die Funktion `dequeue` aufruft. Hinterher werden die Zeiger `head`, `tail` und `nexthead` initialisiert. Danach wird im `head` der Wert des globalen `Head`, im `tail` der Wert des globalen `Tails` gelesen und dann im `nexthead` wird der Wert von `head->next` gelesen. Nachfolgend falls `head` noch konsistent ist, also `head` und `tail` sind gleich, `nexthead` `NULL` ist, dann ist die Queue leer und die funktion gibt `false` zurück. Ansonsten wird der globalen `Head` durch `nexthead` ersetzt und endlich wird der alte `head` freigegeben.

## 2.5 Analyse von Funktionalität <F50>: <Insert>

<F50>: Für die Insertfunktion muss die Liste initialisiert vorliegen. Der User ruft dann die Funktion `insert` auf, die eine Node mit gegebenem Key in das Set an die richtige Stelle einfügt. Im erstem Schritt wird eine neue Node alloziert und initialisiert. Danach begibt sich die Funktion in einen Loop, der läuft bis der CAS gelingt, darauf wird gleich noch genauer eingegangen. Zuerst wird mit `Search` die linke und rechte Node zum passenden Key rausgesucht und geladen. Danach wird `newnode->next`, die Folgenode zu `newnode` die momentan noch auf `NULL` zeigt, auf `rightnode` gesetzt und `rightnode` wird fortan nicht mehr benötigt. Mit CAS wird hier der Compare-and-Swap gemeint der in 1.1 beispielsweise schon genauer beleuchtet wurde. Mit diesem wird `leftnode->next` atomar gesetzt und die Node, falls erfolgreich, ist nun eingefügt. Die `leftnode` wird nun auch nicht mehr benötigt und muss in einer neuen iteration neu geladen werden. Falls der CAS fehlschlägt beginnt der Loop von vorne. Wenn der CAS erfolgreich ist wird 1 returned und die sowohl die `newnode` als auch die Liste ist nicht mehr in benutzung.

## 2.6 Analyse von Funktionalität <F60>: <Delete>

<F60>: In diesem Sequenzdiagramm, wird der Prozess des delete in Set dargestellt. Dies geschieht indem der User die Funktion `delete(value)` aufruft. Hinterher werden lokale Objekte

rightnode, rightnodenext, leftnode und tmp erstellt, um die alten und die neuen Nodes zu halten. Danach wird die Funktion Search in Set, die unten erklärt ist, auf rightnode mit den leftnode und gegebene value durchgeführt. Danach falls die Value von rightnode unterschiedlich ist zu dem gesuchten value oder falls rightnode gleich tail der Liste ist, dann gibt die Funktion 0 zurück. Denn dann hat search keine passende Position für unsere Value im Set gefunden. Falls nicht, dann wird rightnodenext die next value von rightnode bekommen. tmp wird dann auf rightnode gesetzt, tmp wird als logisch gelöscht markiert und danach wird rightnodenext auf tmp gesetzt. Anschließend wird rightnode physisch gelöscht mithilfe von der search Funktion. Letztendlich wird der reference counter decrementiert und rightnode freigegeben.

## 2.7 Analyse von Funktionalität <F70>: <Find>

<F70>: In diesem Sequenzdiagramm, wird der Prozess des Find in Set dargestellt. Dies geschieht indem der User die Funktion find(value) aufruft. Hinterher werden die lokalen Objekte leftnode und rightnode erstellt. Danach wird die Funktion Search in Set, die unten erklärt ist, auf rightnode mit den leftnode und gegebene value durchgeführt. Danach wird kontrolliert ob Search eine Valide Position bestimmt hat und dementsprechend 0 oder 1 returned.

## 2.8 Analyse von Funktionalität <F80>: <Search>

<F80>: Der Search Algorithmus lädt in 3 Verschiedenen Abschnitten verschiedene Objekte. Zuerst werden leftnext und rightnode angelegt, danach begibt sich die Funktion in einen Loop der läuft bis ein return gegeben wird. Im ersten Teil des Loops werden zwei helper Nodes temp und tempnext angelegt und dann auf den Anfang der Liste gesetzt. Diese temporären Nodes sind für das iterieren durch die Liste gedacht, was im nächsten Loop geschieht. Innerhalb des Loops werden left und leftnext auf die temporären Nodes gelegt, solange bis der Key einer unmarkierten temporären Node größer ist als der zu suchende Key. Nach dem Loop wird rightnode auf temp gesetzt, ab jetzt werde temp und tempnext nicht mehr benötigt. Falls weiterhin keine markierte Node zwischen den beiden liegt kann rightnode returned werden. Falls zwischendurch doch Nodes zwischen left und right markiert wurden, wird mithilfe eines CAS die betroffene Node ausgehängt und anschließend vom Speicher entfernt. Letztendlich wird nochmals überprüft ob keine Veränderung in der Liste stattgefunden hat und falls ja wird wieder die rightnode returned.

## 2.9 Analyse von Funktionalität <F90>: <Visualisieren>

<F90>: In diesem Sequenzdiagramm, wird der Prozess der Visualisierung dargestellt. Dies geschieht, indem der User das UserInterface startet. Das UserInterface erstellt dann ein neues Objekt der Klasse LineChart, welches unser Diagramm generiert. LineChart öffnet dann die .csv Datei mit den Benchmark results, die dann returned werden und generiert mit den Daten den gewünschten Graphen der dann dem User returned wird.

## 2.10 Analyse von Funktionalität <F100>: <Benchmark>

<F100>: Bei dem Benchmark wird eine .csv Datei angelegt und geöffnet, die dann die Ergebnisse beinhalten soll. Die gewünschte Anzahl an Threads werden angelegt und dann mit der Benchmark Funktion aufgerufen. Diese Funktion berechnet die Gesamtzeit und die Latenz für die gewünschte Anzahl der Operationen Opreationen. Sobald die Threads mit der ermittlung fertig sind werden die Ergebnisse returned und die Threads werden aufgelöst. Anschließend werden die Ergebnisse in die Datei geschrieben und diese dann geschlossen. Schlussendlich returned der Benchmark die Datei für die Visualisierung.

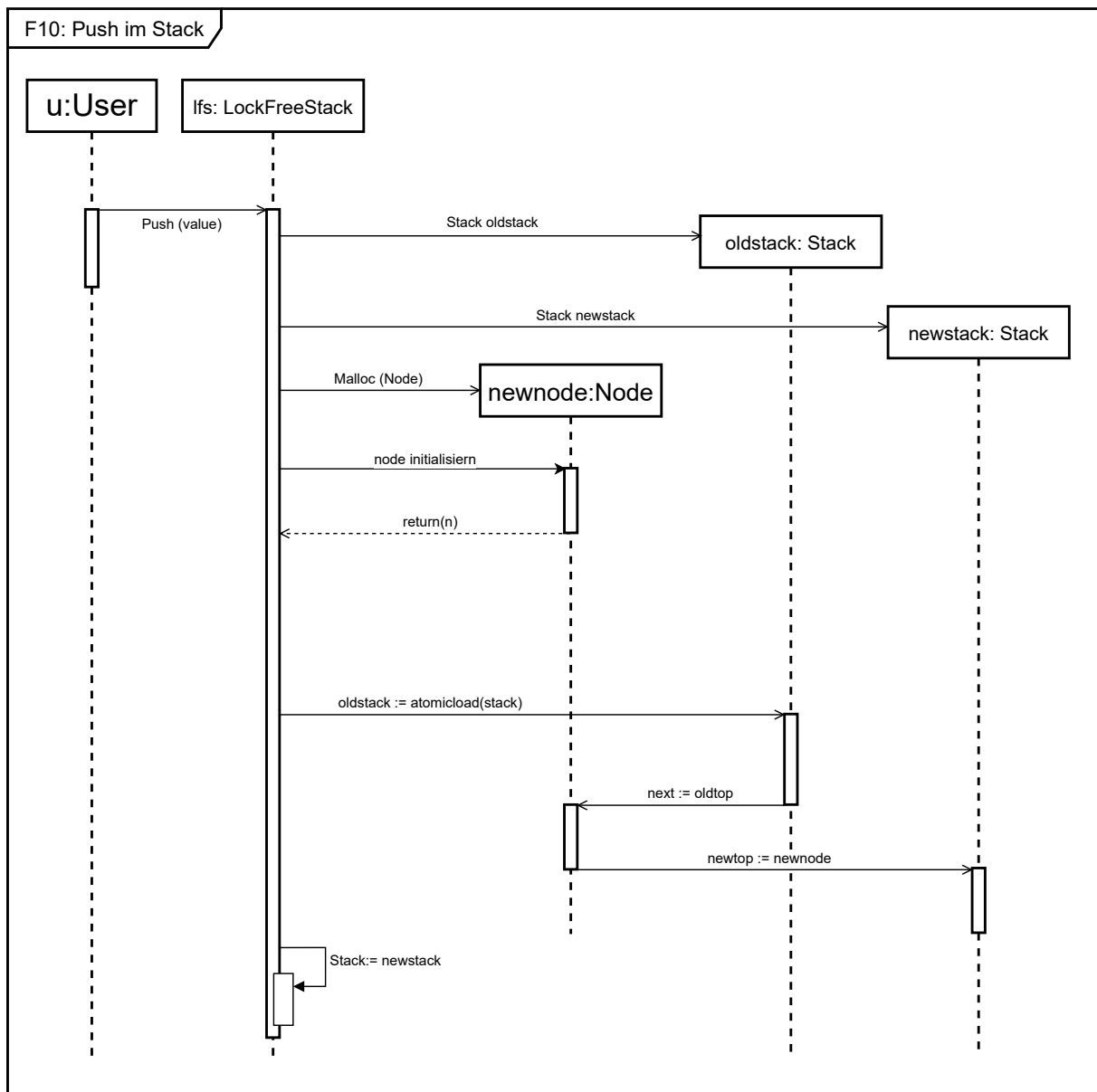


Abbildung 2.1: Pushen im Stack

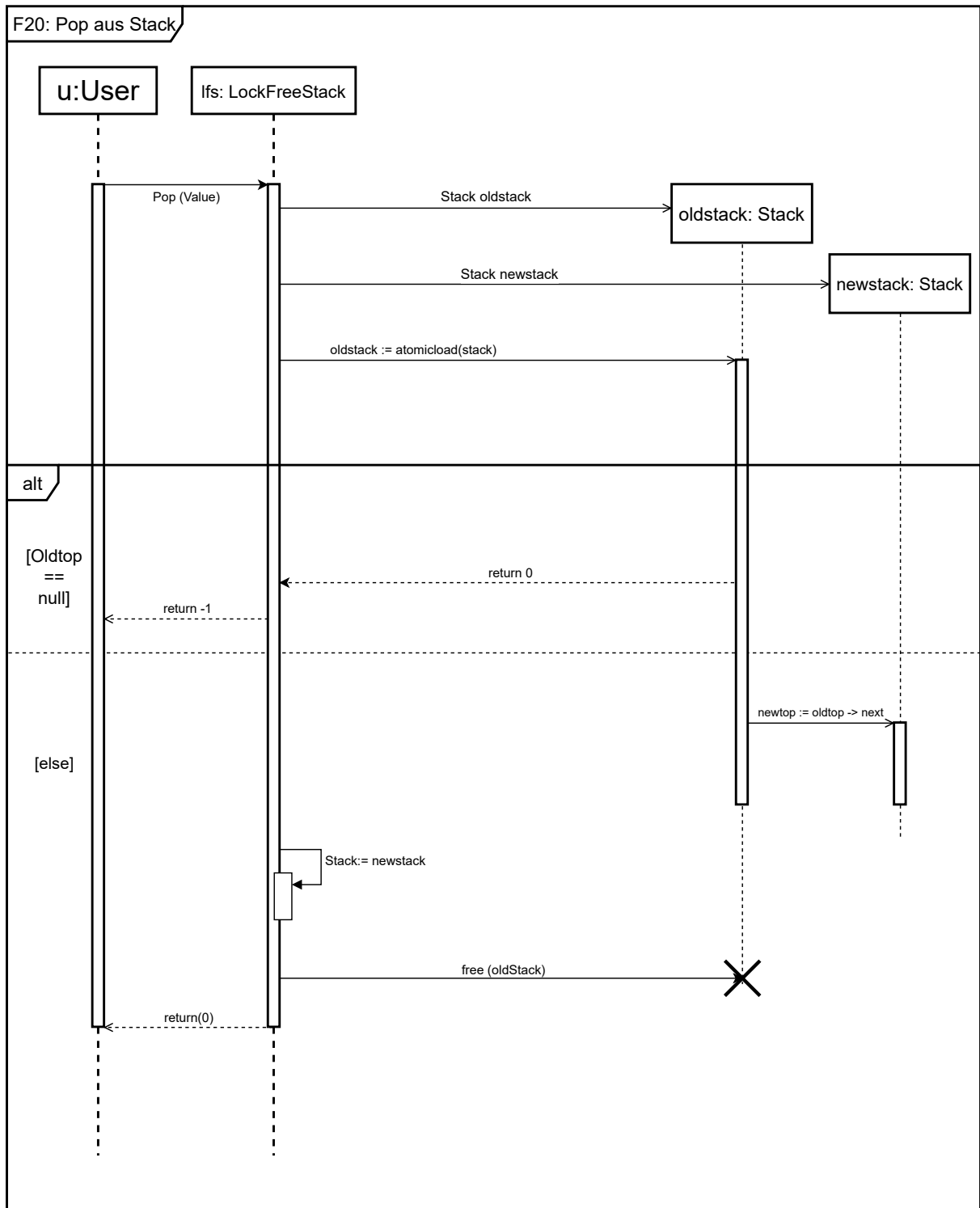


Abbildung 2.2: Pop aus Stack

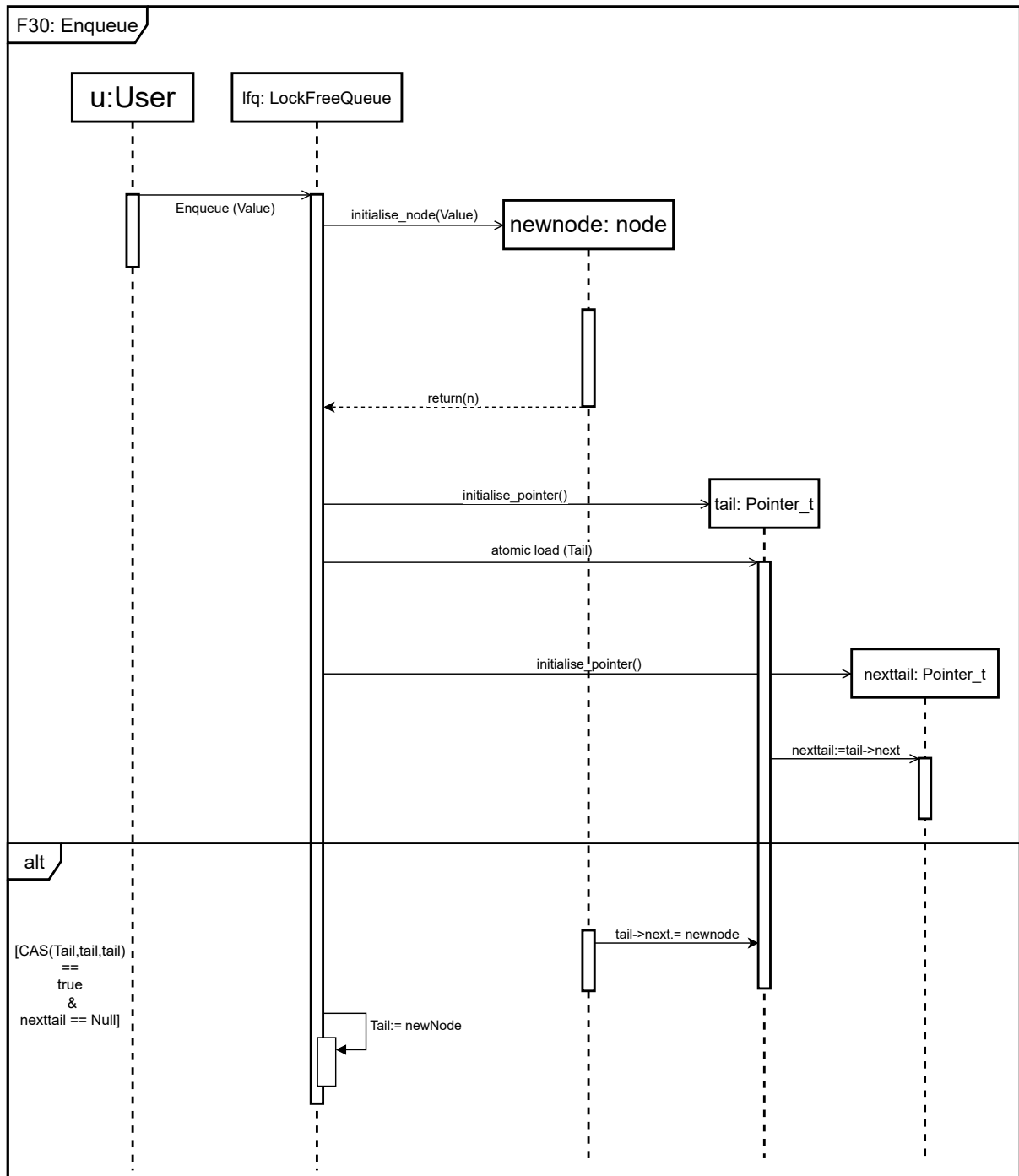


Abbildung 2.3: Enqueue



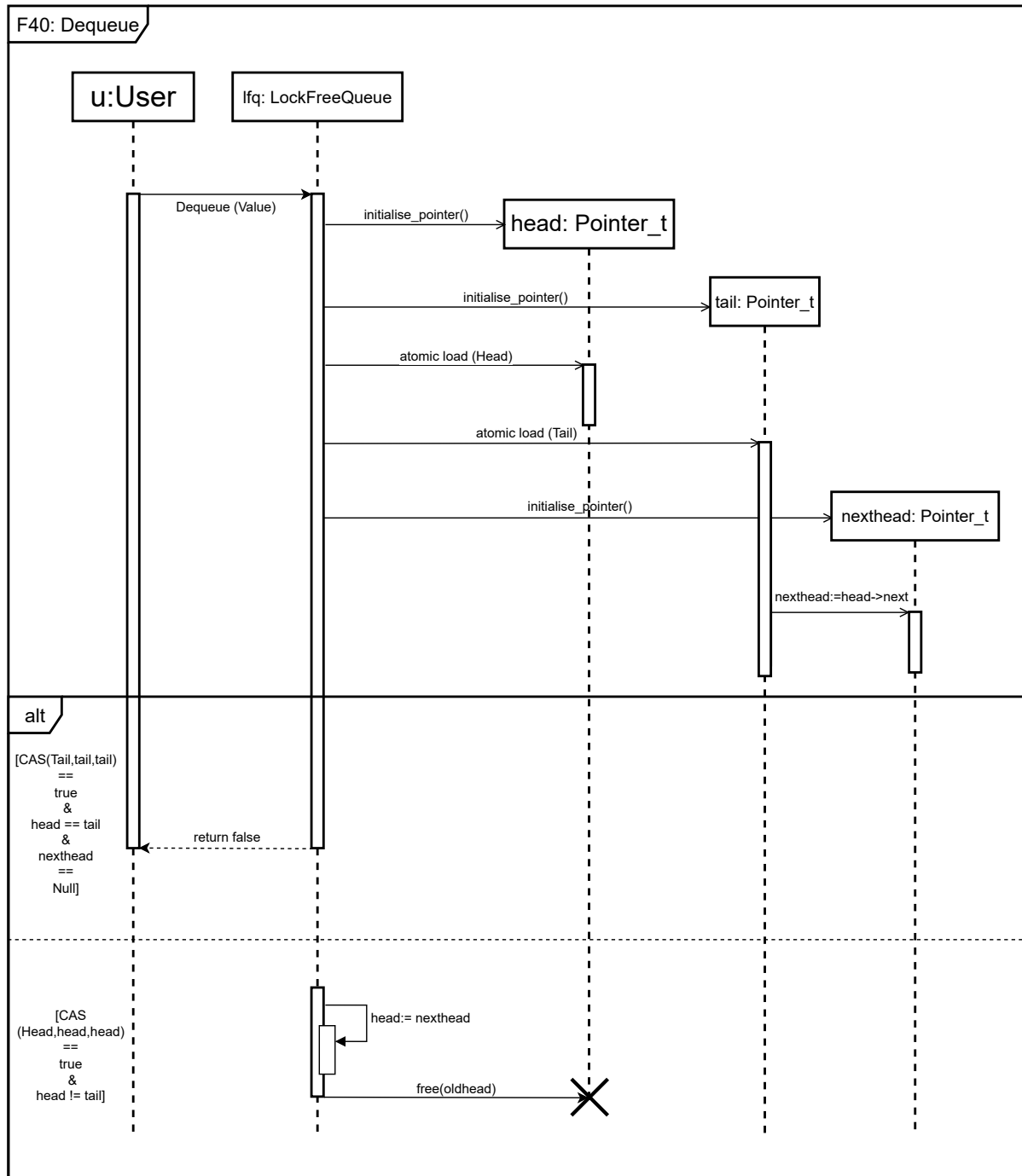


Abbildung 2.4: Dequeue

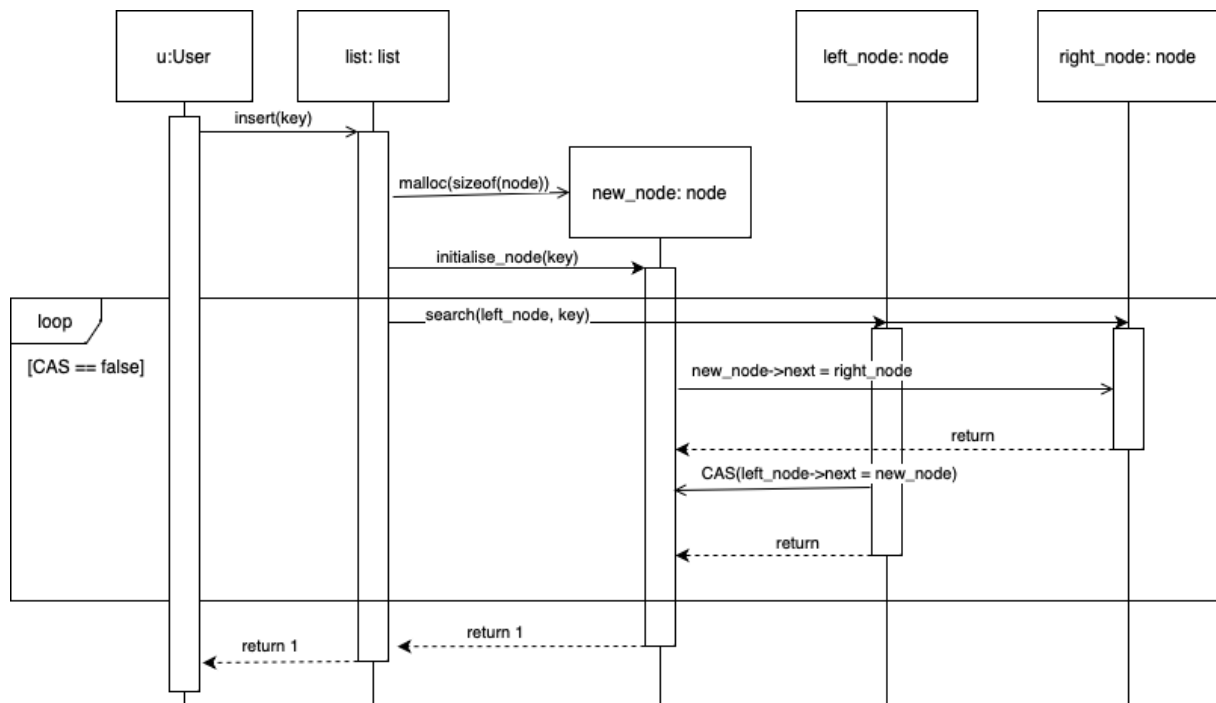


Abbildung 2.5: Insert im Set

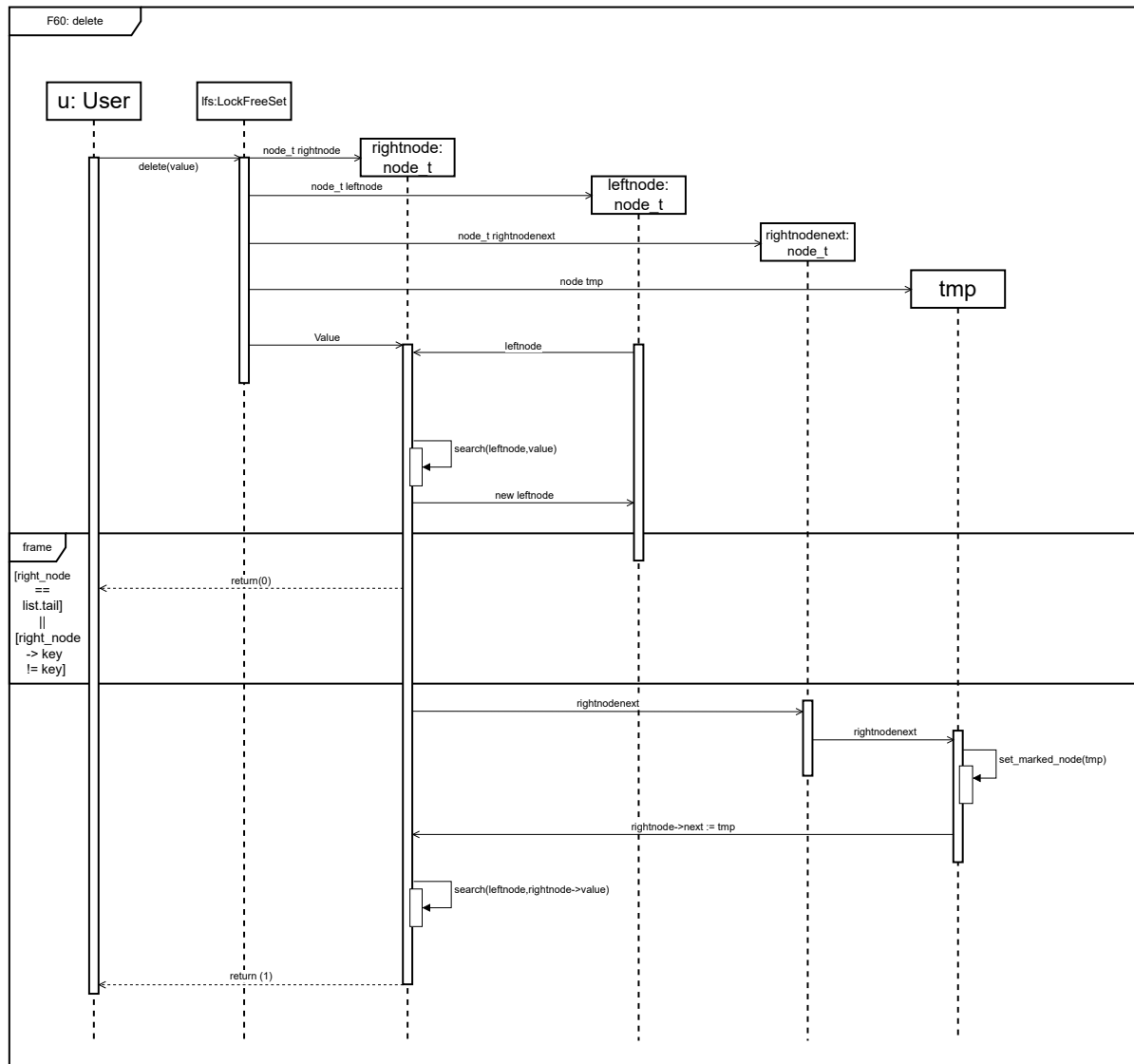


Abbildung 2.6: Delete

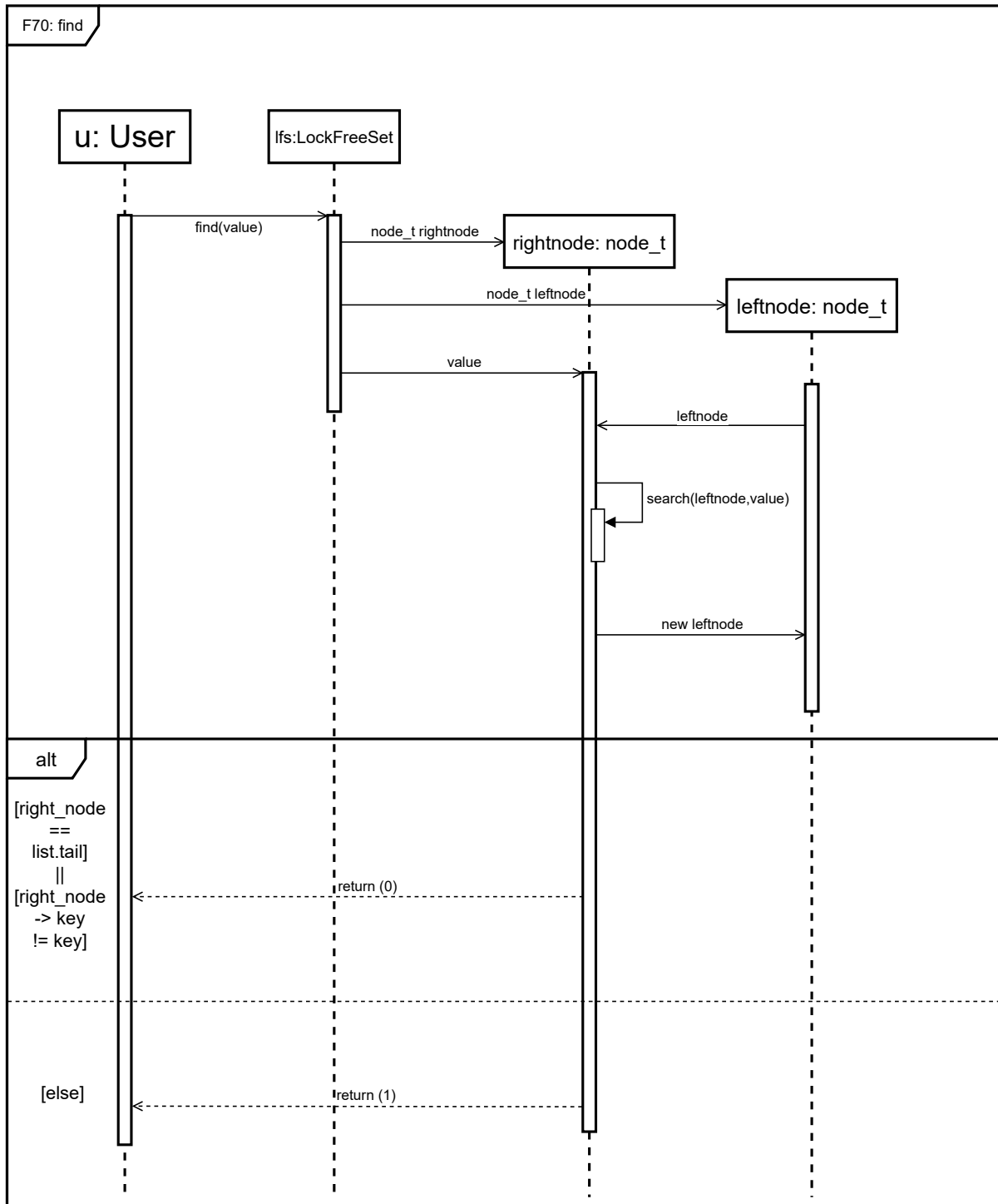


Abbildung 2.7: Find

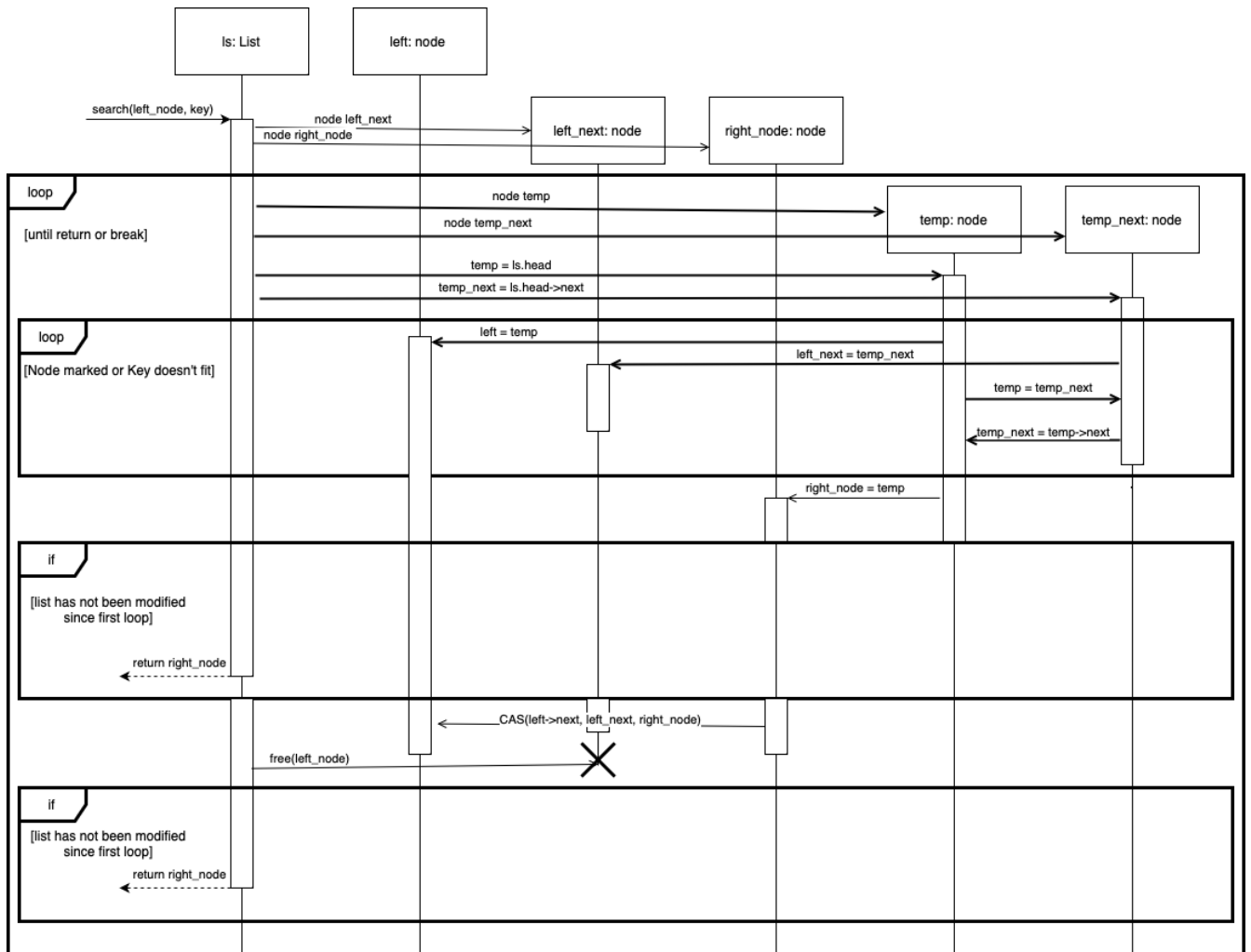


Abbildung 2.8: Search in Set

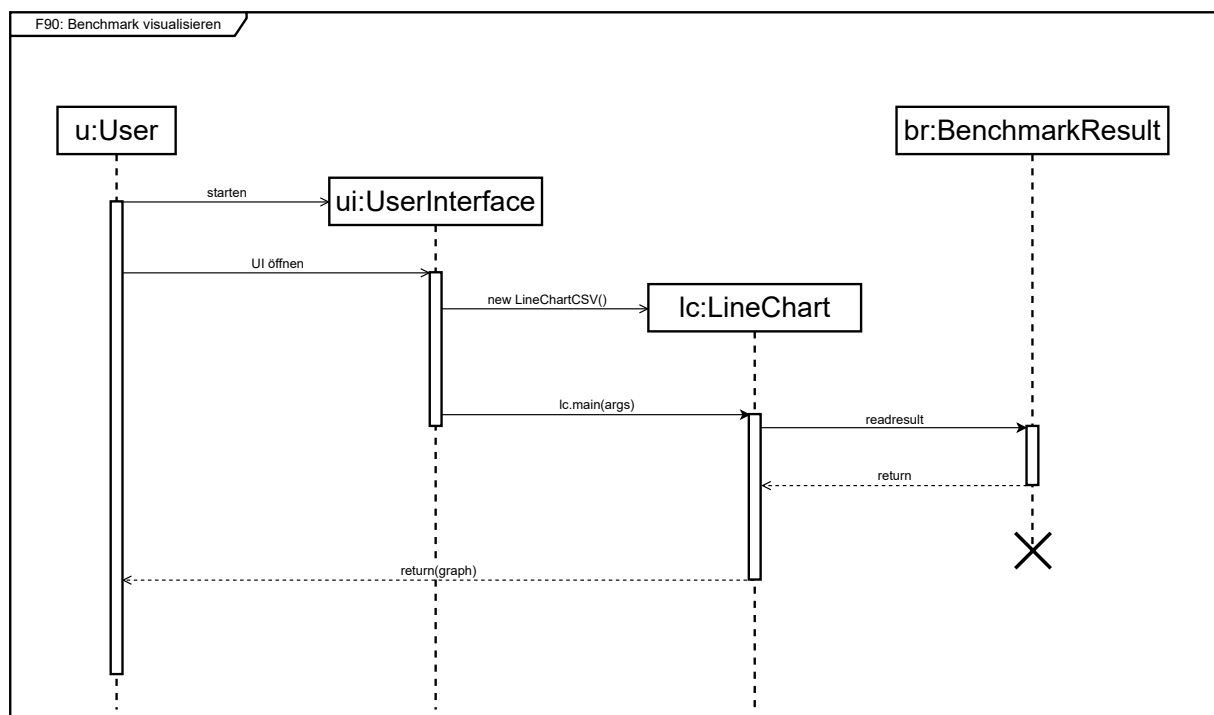


Abbildung 2.9: Visualisieren

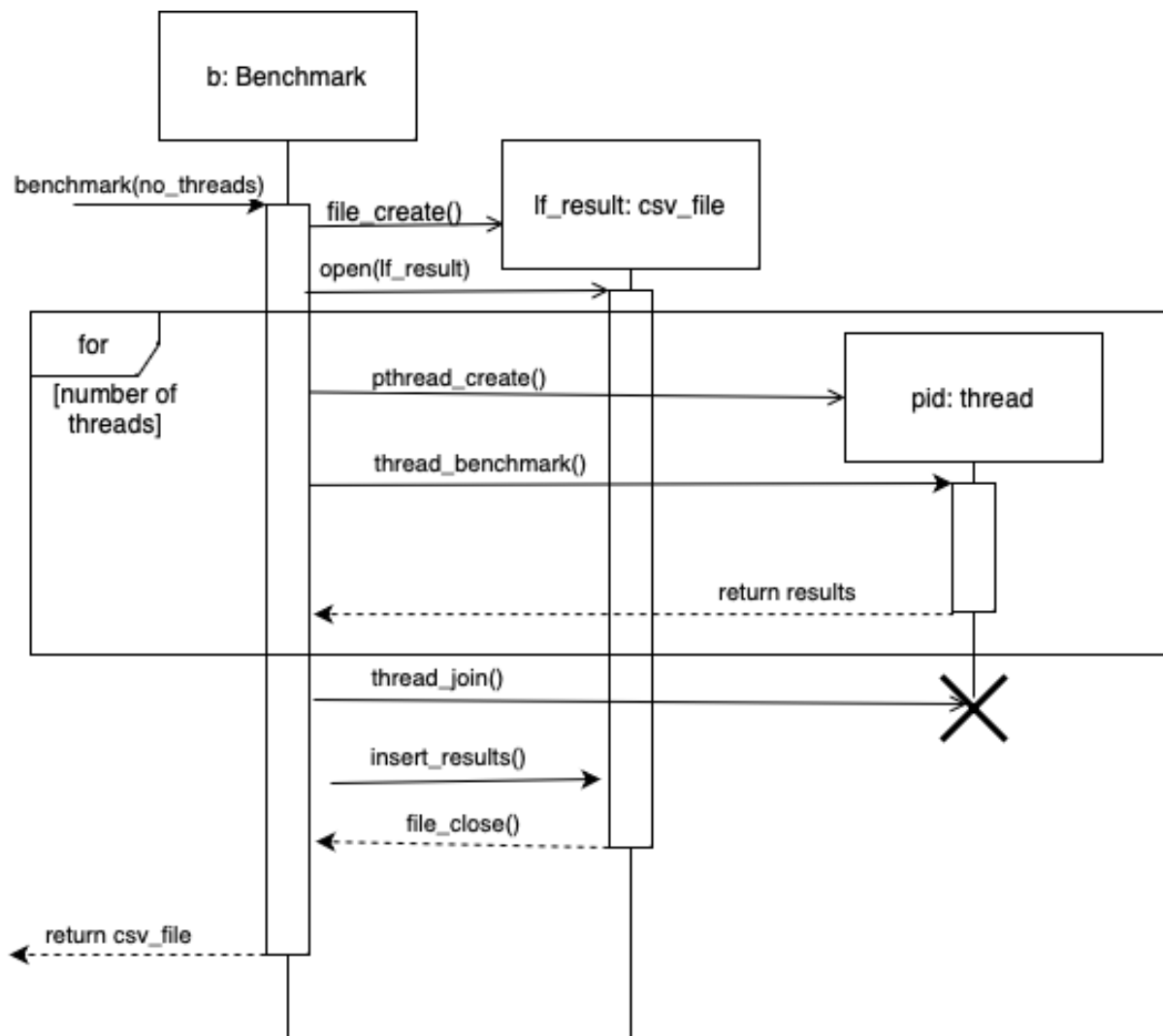


Abbildung 2.10: Benchmark

## 3 Datenmodell

In diesem Kapitel sollen dauerhaft gespeicherte Daten beschrieben werden.

### 3.1 Diagramm

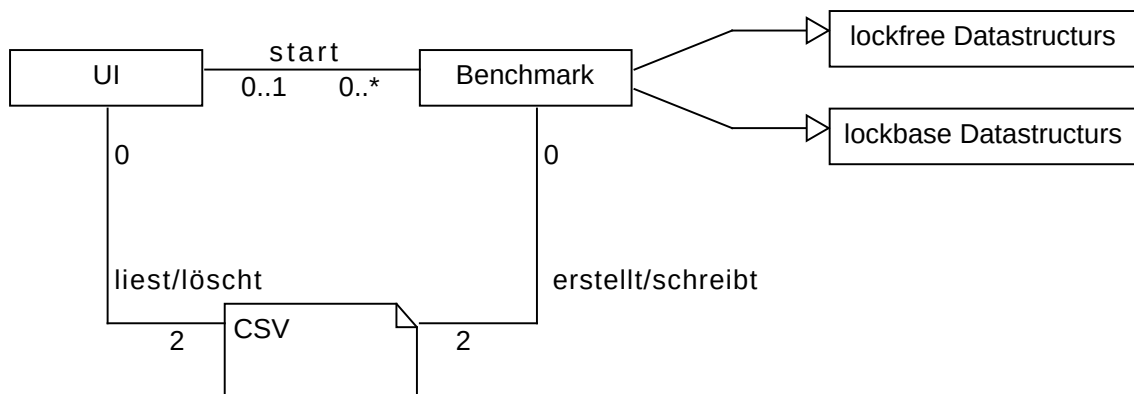


Abbildung 3.1: Diagramm zur Erstellung der Benchmark Ergebnisse

### 3.2 Erläuterung

Das Klassendiagramm enthält keine Entities, da es keine dauerhaft gespeicherten Daten gibt. In den Datenstrukturen werden keine Daten dauerhaft gespeichert. Bei dem Benchmark werden Daten automatisch erstellt und in die ausgewählte Datenstruktur eingefügt bzw. gelöscht. Dabei werden Daten gemessen und in einer temporären CSV-Datei gespeichert. Die UI liest und virtualisiert die Daten der CSV-Datei und löscht die Datei anschließend.

Es gibt somit keine Daten, die dauerhaft von der Anwendung gespeichert werden.



## 4 Konfiguration

In diesem Kapitel werden sämtliche Konfigurationen genannt, die für die Erfüllung des Softwareprojekts notwendig sind.

Das Projekt besteht dabei aus zwei Teilen, drei vollständig implementierten Datenstrukturen und deren Visualisierung.

Um die Verwendung der Datenstrukturen für möglichst viele Personen zu ermöglichen, benötigt der Computer lediglich eines der drei am häufigsten verwendeten Betriebssysteme, Windows, Mac OS oder LINUX. Es wird keine Internetverbindung benötigt um die Visualisierung oder die Library zu benutzen. Um die Library in ein eigenes Programm einzubinden muss sie lediglich heruntergeladen und importiert werden, auch hier sind keine besonderen config Dateien nötig.

Zur Ausführung der Visualisierung wird eine Java Installation benötigt und muss gegebenenfalls vom Nutzer installiert werden. Für die Visualisierung muss auch gegebenenfalls JavaFX installiert und aufgesetzt werden.

## 5 Glossar

Hier werden Fachbegriffe erklärt.

| Begriff                           | Definition                                                |
|-----------------------------------|-----------------------------------------------------------|
| Aktivitätsdiagramm                | Graphische Darstellung von Software                       |
| Allozieren                        | Zuweisen von Speicherplatz                                |
| Atomare Operationen               | Operation nur als Ganzes erfolgreich ist oder fehlschlägt |
| Betriebssystem                    | Zusammenarbeit von Software und Hardware                  |
| Compare and Swap (CAS)            | Atomare Vergleichsoperation                               |
| Datenstruktur                     | Objekt, in dem Daten gespeichert werden                   |
| Deadlock                          | mehrere Prozesse blockieren sich gegenseitig              |
| Hardwarekomponenten               | Physische Komponenten eines Systems                       |
| Implementieren                    | Erzeugen eines funktionsfähigen Programms                 |
| JavaFX                            | Framework zur Erstellung von Java-Applikationen           |
| Library                           | Sammlung ähnlicher Objekte                                |
| Linux                             | Ein freies Open Source-Betriebssystem                     |
| List                              | Datenstruktur                                             |
| Lockfrei                          | Ohne Verwendung von Locking Algorithmen                   |
| Mac OS                            | Von Apple entwickeltes Betriebssystem                     |
| Mehrkernsysteme                   | System mit mehreren Prozessor-Kernen                      |
| Node                              | Element/Knoten                                            |
| Queue                             | Datenstruktur                                             |
| Sequenzdiagramm                   | Graphische Darstellung von Interaktionen                  |
| Set                               | Datenstruktur                                             |
| Software                          | Computerprogramme                                         |
| Thread                            | Aktivitätsträger in der Abarbeitung eines                 |
| Programms (Teil eines Prozessors) |                                                           |
| Thread-Sicherheit                 | Konsistenz auch bei Mehrkernbetrieb                       |
| Visualisierung                    | IdR Daten und Zusammenhänge graphisch darstellen          |
| Wettlaufsituation                 | Ergebniss hängt von Prozessreihenfolge ab                 |
| Windows                           | Von Microsoft entwickeltes Betriebssystem                 |